

## PRZEWODNIK PO WALIDACJI (ETAP 4)

### DOBRE PRAKTYKI I METODYKA WERYFIKACJI SYSTEMU W WERSJI 1.0

Etap 4 ma jedno nadrzędne zadanie: **bezpornie i obiektywnie udowodnić, że finalny program działa i spełnia założenia lub NIE w różnych zakresach.**

Proces walidacji nie polega na dalszym "programowaniu" czy "poprawianiu" błędów. Zbudowany program w wersji 1.0 zamrażamy, a naszą rolą staje się rola audytora.

Proces walidacji będzie stanowić istotny rozdział dokumentacji końcowej projektu.

Zwieńczeniem tej pracy będzie Seminarium, na którym poza pełną prezentacją projektu pokażecie wszystkim wyniki walidacji.

Poniższy przewodnik przedstawia zbiór dobrych praktyk, które pomogą Wam rzetelnie przeprowadzić walidację Waszego algorytmu.

---

## ŚRODOWISKO I DANE TESTOWE

Najpoważniejszym błędem metodycznym, jaki można popełnić na etapie walidacji, jest testowanie algorytmu na tych samych danych, które posłużyły do jego stworzenia (treningu) lub kalibracji w Etapie 2 i 3.

### 1. Niezależny zbiór testowy (Test Set)

Walidacja wersji 1.0 **musi zostać przeprowadzona wyłącznie na danych, których algorytm (i Wy) nigdy wcześniej nie "widział"**.

- Jeśli Wasz system zlicza wolne miejsca w SKS, nagrajcie zupełnie nowy materiał wideo w innym dniu lub o innej porze.
- Tylko niezależny zbiór pozwala na rzetelną ocenę tego, czy system faktycznie potrafi generalizować problem, czy jedynie "nauczył się na pamięć" wcześniejszych próbek.

### 2. Reproduktywność warunków

Należy szczegółowo przeanalizować i odtworzyć warunki brzegowe zbioru, takie jak różnice w oświetleniu, rozdzielczość, okluzje (zasłonięcia obiektów) czy perspektywa. Środowisko testowe powinno być powtarzalne. Zadbajcie o to, aby Wasz test uwzględniał całe spektrum trudności danych badawczych – od przypadków trywialnych, po skrajnie trudne warunki, jak np. noc, deszcz, czy niska jakość wejścia.

### 3. Rzetelne adnotowanie nowego zbioru testowego

Podobnie jak w Etapie 2, jakość wyników ewaluacji zależy wprost od jakości i dokładności danych wejściowych. Nowy zbiór danych, na którym przeprowadzicie walidację, musi zostać

starannie oznaczony (labelowanie). Skrupulatne przygotowanie tzw. "Ground Truth" (czyli idealnych, oczekiwanych wyników) jest niezbędne do wyliczenia błędu Waszego programu.

---

## EWALUACJA PRZY UŻYCIU "TWARDYCH" METRYK

Słowo "działa dobrze" nie jest pojęciem inżynierskim. Walidacja wersji 1.0 wymaga posługiwania się wyłącznie twardymi danymi i metrykami.

### 1. Metryki poprawności klasyfikacji i detekcji

Wyjście z programu musi bezpośrednio zasilać Waszą dokumentację liczbową. Program musi wygenerować dane, które pozwolicie poddać krytycznej ocenie. Do najważniejszych miar, które bezwzględnie powinny znaleźć się w walidacji, należą:

- **Precyzja (Precision):** Jaki procent obiektów rozpoznanych przez system jako pozytywne faktycznie taki jest? Metryka kluczowa tam, gdzie fałszywy alarm jest kosztowny.
- **Czułość (Recall):** Jaką część wszystkich faktycznych obiektów w zbiorze system zdołał poprawnie zidentyfikować?
- **F1-Score:** Średnia harmoniczna z precyzji i czułości, stanowiąca doskonałą metrykę dla niezbalansowanych zbiorów danych.
- **IoU (Intersection over Union):** Podstawowa metryka w projektach wizyjnych opartych na klasycznych detektorach, sieciach YOLO czy ResNet. Pozwala ocenić, jak dokładnie wygenerowany *bounding box* (ramka otaczająca) pokrywa się z faktycznym obiektem na zdjęciu.

### 2. Metryki wydajnościowe

Walidacja to nie tylko skuteczność, ale też wydajność i szybkość. Zmierzcie wskaźnik **FPS (Frames Per Second)**, zwłaszcza jeśli projekt dotyczy analizy strumieni wideo w czasie rzeczywistym. Należy wyraźnie wskazać, czy system osiągnął założoną płynność i jakie są wymagania sprzętowe niezbędne do jego działania. Zidentyfikujcie i opiszcie, czy nadal występują jakieś wąskie gardła w zużyciu zasobów pamięci.

---

## ZARZĄDZANIE PRZYPADKAMI BRZEGOWYMI I ANALIZA BŁĘDÓW

Prawdziwe R&D poznaje się po tym, jak system i inżynierowie radzą sobie z porażkami. Etap walidacyjny na pewno obnaży pewne niedoskonałości wersji 1.0. Ważne, by je metodycznie sklasyfikować.

### 1. Analiza False Positives i False Negatives

W Etapie 4 wciąż nie wystarczy stwierdzić "program wykrywa dym" – musicie zliczyć i przeanalizować wszystkie fałszywe alarmy (False Positives) oraz przypadki braku detekcji (False Negatives).

- **Wizualizacja błędów:** Koniecznie załączcie do raportu z walidacji rzuty ekranu z sytuacjami, w których system zawiódł w ostatecznym teście (np. cień potraktowany jako hulajnoga lub brak detekcji na jasnym tle).
- **Wnioskowanie analityczne:** Opiszcie, z czego wynikają te błędy (np. zbyt mały kontrast, wada samego zbioru testowego, ograniczenia wybranej architektury YOLO).

## 2. Konfrontacja wyników z celami Etapu 1

Zestawcie wyniki otrzymane na niezależnym zbiorze walidacyjnym z Waszymi pierwotnymi założeniami z Etapu 1. Czy obietnice postawione w pierwszych tygodniach kursu znalazły pokrycie w wynikach wersji 1.0? Jeśli w wyniku problemów natury badawczej i zmiennych poddawanych testom skuteczność jest niższa niż zakładaliście – uzasadnijcie dlaczego (np. parametry progowania lub morfologii zawiodły w specyficznych warunkach oświetleniowych).

---

## WPŁYW ZAAWANSOWANEGO PRZETWARZANIA NA KOŃCOWY WYNIK (ABLATION STUDIES)

W ramach dobrych praktyk walidacyjnych, warto przeprowadzić ostateczne podsumowanie optymalizacji algorytmu.

**Testy parametryczne i wizualizacja logiki:** Niezależnie od wybranej metody (klasyczna analiza obrazu, detektory Haar Cascades/HOG, czy sieci neuronowe), na ostateczny wynik mają wpływ parametry takie jak rozmiar jądra filtra, próg detekcji czy współczynnik uczenia. Dobrą praktyką na tym etapie jest zaprezentowanie, np. w ujęciu tabelarycznym, zestawienia porównującego działanie ostatecznej, "najlepszej" konfiguracji systemu wersji 1.0 na tle konfiguracji domyślnych. Pokażcie graficzną reprezentację (zrzuty ekranu, maski segmentacji) udowadniającą, że wejściowy obraz po przejściu przez ustalony, finalny *Pipeline* algorytmu skutkuje zoptymalizowanym, poprawnym wynikiem.